

# SGLang-Omni 每日 PR 学习日报

日期：2026-06-03 | 仓库：[sgl-project/sglang-omni](#) | 统计窗口：过去 24 小时内已合并 (merged) 的 PR

## 当日概览

5

合并 PR 总数

1

核心架构 PR (重点精讲)

3

文档 / Logo / License

1

CI / 测试清理

主题分类：今天唯一一个“硬核”改动是 **核心架构** **#609 — Add MOSS-TTS (OpenMOSS) support**：把一个全新的**多阶段 (multi-stage) 流式 TTS 模型**原生接入 SGLang-Omni 的 **OmniScheduler** 三段式流水线。其余四个是文档 / Logo / License / 测试清理类小改动。按照「深度优先于广度」的要求，本日报把 90% 的篇幅用于**把 #609 讲透**，其余 PR 一笔带过。

| #                    | 标题                                     | 作者          | 分类    | 讲解深度 |
|----------------------|----------------------------------------|-------------|-------|------|
| <a href="#">#609</a> | Add MOSS-TTS (OpenMOSS) support        | GaikaiZhang | 核心架构  | 深度精讲 |
| <a href="#">#669</a> | [Doc] Add MOSS-TTS in SGLang-Omni docs | gaoyang07   | 文档    | 简述   |
| <a href="#">#668</a> | Remove stale S2-Pro docs test          | xinlij      | CI/测试 | 简述   |
| <a href="#">#671</a> | Docs/svg logo (暗色模式适配)                 | Oxjeffro    | 文档    | 简述   |
| <a href="#">#650</a> | Docs/license                           | xinlij      | 文档    | 简述   |

作者：GaokaiZhang 状态：merged (2026-06-02 由 zhaochenyang20 合并)

PR #609 +3617 / -46, 24 个文件, 23 次提交

**一句话概括：**把开源语音合成模型 MOSS-TTS (OpenMOSS 出品) 原生接入 SGLang-Omni，实现成一条 预处理 → SGLang 自回归引擎 → MOSS 声码器 的三阶段流水线；核心难点是「**一步要同时吐出 1 个文本 token + N 个音频码本 token、而且各码本之间有时间错位 (delay pattern)**」的多头模型，跑在原本为「一步一个 token」设计的 SGLang 高性能推理引擎上。

## 1. PR 概览

- **标题：**Add MOSS-TTS (OpenMOSS) support
- **编号 / 作者：**#609 / GaokaiZhang (贡献者)
- **状态：**已合并 (merged into `main`)
- **链接：**<https://github.com/sgl-project/sglang-omni/pull/609>

这是一个典型的「**接入一个新模型**」型 PR，但 MOSS-TTS 的结构特殊，所以它实际上是在 SGLang-Omni 的多阶段框架里搭了一整条新流水线，并对 SGLang 的采样 / 解码循环做了不少“魔改”。

## 2. 它解决了什么问题

先理解背景：**TTS (Text-To-Speech, 文本转语音)** 就是“输入一段文字，输出一段说话的音频”。现代 TTS 大多走「**语言模型 + 声码器**」的两段式套路：

- 第一段：一个像 GPT 一样的**自回归语言模型**，不过它预测的不是文字，而是一串“**音频 token**”（离散化的声音编码，下面会解释 RVQ）。
- 第二段：一个**声码器 / 解码器 (vocoder)**，把这串音频 token 还原成真正能听的波形（24kHz 的 wav）。

**痛点在哪？**普通的文本大模型，自回归生成时是“一个时间步预测一个 token”。但 MOSS-TTS 这类高质量 TTS 用了一种叫 **RVQ (残差矢量量化)** 的音频编码：每一帧声音不是 1 个 token，而是 **N 个码本 (codebook) token 叠在一起**（这里 N 可达 32）。也就是说模型每一步要**同时**输出「1 个文本通道 + N 个音频码本通道」共 33 个 token。更麻烦的是，它用了 **delay pattern (延迟错位)**：第 k 个码本要比第 0 个码本晚 k 步才开始生成。

而 SGLang (SGLang-Omni 底层的高性能推理引擎) 从设计上就假设“**每个请求每一步只产出 1 个 token**”，它的 CUDA Graph、KV cache、radix cache、连续批处理全都围绕这个假设构建。**所以问题就是：怎么把“一步**

33 个 token + 通道间错位”的怪模型，塞进“一步 1 个 token”的高性能引擎里，还要享受到引擎的批处理 / CUDA Graph / 缓存加速？这正是本 PR 要解决的核心矛盾。

### 3. 具体做了什么改动

#### 3.1 涉及的核心文件

|                                                             |       |     |
|-------------------------------------------------------------|-------|-----|
| sglang_omni/models/moss_tts/stages.py                       | +394  |     |
| sglang_omni/models/moss_tts/model_runner.py                 | +558  |     |
| sglang_omni/models/moss_tts/sglang_model.py                 | +478  |     |
| sglang_omni/models/moss_tts/request_builders.py             | +689  |     |
| sglang_omni/models/moss_tts/codec.py                        | +70   |     |
| sglang_omni/models/moss_tts/config.py /<br>payload_types.py | +169  |     |
| tests/unit_test/moss_tts/test_pipeline.py                   | +908  |     |
| docs/cookbook/moss_tts.md + benchmarks + 配置/接线              | +~350 | -46 |

#### 3.2 三阶段流水线是怎么定义的 (config.py)

整条流水线在 `config.py` 里被声明成三个 `StageConfig`，这就是 SGLang-Omni「多阶段架构」的体现——一个模型不是一个黑盒，而是一串可独立调度的 stage：

```
class MossTTS PipelineConfig(PipelineConfig):
    # 流水线: 预处理 -> AR引擎 -> 声码器
    stages = [
        StageConfig(name="preprocessing", # 阶段1: CPU上做分词+编码参考音频
                    factory="...stages.create_preprocessing_executor",
                    next="tts_engine"),
        StageConfig(name="tts_engine", # 阶段2: GPU上跑SGLang自回归引擎
                    factory="...stages.create_sglang_tts_engine_executor",
                    gpu=0, next="vocoder"),
        StageConfig(name="vocoder", # 阶段3: GPU上把音频码还原成波形
                    factory="...stages.create_vocoder_executor",
                    gpu=0, terminal=True), # terminal=最后一段
    ]
```

每个 stage 有自己的 `factory`（工厂函数，负责创建这个阶段的执行器）和 `next`（下一阶段名字），数据以 `StagePayload` 的形式在阶段间流动。这种设计的好处：三段可以各自用最合适的并发策略、跑在不同设备上、独立扩缩容。

### 3.3 阶段一：预处理（为什么要多并发）

预处理做两件 CPU 重活：把文字分词、把“参考音频”（用于音色克隆）编码成音频 token。这里有个很值得学的工程决策（`stages.py`）：

```
def create_preprocessing_executor(model_path, *, max_concurrency=8):
    processor = _load_moss_processor(model_path, device="cpu", ...)
    # 预处理是CPU密集型: 每个请求都要给参考音频跑一遍codec编码。
    # 如果串行(并发=1), codec编码会霸占时间, 把下游AR引擎饿到batch=1,
    # 而batch=1正是实时率(RTF)最差的情况。所以并行跑多个——
    # torch在codec前向时会释放GIL, 让AR引擎收到稳定可批处理的请求流。
    return SimpleScheduler(preprocess_moss_tts_payload, max_concurrency=max_concurrency)
```

翻译：慢的预处理如果排队串行做，会让快的 GPU 引擎没活干（batch 掉到 1）。所以让预处理开 8 路并发，喂给 GPU 一条“稳定、可成批”的请求流，从而把 GPU 利用率拉满。

### 3.4 阶段二的模型：33 个通道的嵌入与 33 个输出头（sglang\_model.py）

核心模型类 `MossTTSDelaySGLangModel` = Qwen3 主干 + 多通道嵌入 + 多通道输出头。与普通 LLM 只有 1 个 embedding 表和 1 个 lm\_head 不同，它有 `channels`（=N+1=33）份：

```
# 33个独立的嵌入表(1个文本通道 + 32个音频码本通道)
self.embedding_list = ModuleList([VocabParallelEmbedding(vocab_size_list[i],
                                                         for i in range(channels))])

self.model = Qwen3Model(config.language_config) # 共享的Transformer主干
# 33个独立的输出头, 每个头的词表大小不同(文本15万 vs 音频1025)
self.lm_heads = ModuleList([ParallelLMHead(vocab_size_list[i], hidden)
                             for i in range(channels)])
```

关键技巧 1：33 个通道如何合成一个输入向量？把每个通道的 token 各自查表，再逐元素相加，得到一个 hidden 向量喂给共享的 Transformer：

```
def _prepare_multi_modal_inputs(self, input_ids):
    # input_ids_2d 形状 [序列长, 33]: 每行是同一时刻的33个通道token
    embeds = torch.zeros(seq_len, hidden)
    for idx, embed_layer in enumerate(self.embedding_list):
        embeds = embeds + embed_layer(input_ids_2d[:, idx]) # 33个嵌入相加
    return embeds
```

类比：普通 LLM 是“一个人说一句话”；MOSS-TTS 是“33 个人（1 个念字幕 + 32 个分别哼不同声部）同时开口，把 33 个声部混音成 1 路信号”送进大脑（Transformer），大脑再分头给出 33 个声部下一拍该怎么唱。

**关键技巧 2：**把 logits 计算移出 `forward`。SGLang 用 CUDA Graph 把模型 forward “录制”成一张固定的计算图来加速（见知识点）。但 33 个头里音频头词表只有 1025，文本头却有 15 万，硬塞进 CUDA Graph 会因为 buffer 复用而出错。作者的做法是：`forward` 只返回 backbone 的 hidden states（套一个 dummy logits 占位），真正的 33 路采样放到图外面由 model runner 做：

```
def forward(self, ...):
    hidden_states = self.model(...) # 只跑被CUDA Graph捕获的主干
    sample_hidden = self._select_sample_hidden_states(hidden_states, fb)
    # 返回dummy logits + 真正的hidden, 让33路采样在图外做,
    # 避免图回放(replay)丢掉customized_info, 也避免复用文本词表的logits buffer
    dummy = sample_hidden.new_empty((sample_hidden.shape[0], 1))
    return LogitsProcessorOutput(next_token_logits=dummy, hidden_states=sample_hi
```

**关键技巧 3：**每个通道用自己词表大小的 LogitsProcessor。这条注释信息量很大：

```
# sglang的 _get_logits 会把头输出切到 config.vocab_size 来去掉
# ParallelLMHead的词表padding(1025 -> 1088)。如果音频头复用文本config
# (词表15万), 那padding列1025..1087不会被切掉, 这些权重≈0的列会在
# 音频温度下被当成"越界码"采样出来 -> 损坏音频帧 -> 拉高WER(字错率)。
channel_config.vocab_size = config.vocab_size_list[channel] # 每通道用自己的
```

这是一个非常实战的 bug 修复思路：张量补齐（padding）本是为了对齐显存，但如果忘了在采样前切掉，补的“垃圾列”会被当成合法 token 采样出来，破坏输出。

### 3.5 阶段二的灵魂：delay-pattern 批量采样（model\_runner.py）

`MossTTSModelRunner._sample_rows` 是整个 PR 最硬核的函数——它把原始 HuggingFace 模型里一个 `for` 循环 + `if/else` 的 `generate` 单步，改写成整批请求一次性、纯张量（向量化）的版本。为什么要向量化？因为 SGLang 是连续批处理，一个 batch 里几十个请求各自处于 delay pattern 的不同阶段（有的还在“铺垫”，有的在正式生成，有的快结束），用 Python `for` 循环逐个处理会成为性能瓶颈。核心状态用一个 `[audio_length, delayed, is_audio]` 三元组表示：

```
# delayed: 这个请求在delay pattern里走到第几步(决定哪些码本该开始生成)
next_text[delayed < n_vq] = delay_slot # 还没铺完: 文本通道填"延迟占位符"
is_audio_eos = delayed == n_vq # 刚好铺完: 该收尾
next_text[is_audio_eos] = audio_end

# 哪些音频码本通道这一步要真正采样? (delay pattern的三角形掩码)
pre_audio = audio_lengths.unsqueeze(1) > channel_idx # 已经"轮到"了
post_audio = channel_idx > (delayed.unsqueeze(1) - 1) # 还没"过期"
sampling_audio_mask = pre_audio & post_audio # 交集=该采样的格子
```

这两行 mask 就是 delay pattern 的精髓：用一个布尔矩阵 `[batch, n_vq]` 一次性标出“这一拍、这个请求、这个码本要不要出声”，全程没有 Python 循环。可复用且可一眼看懂的还有它的采样函数

`_sample_tokens` : 温度 / top-k / top-p 全部支持逐行不同的参数 (每个请求自带采样配置) , 并用 `multinomial_with_seed` 保证无论 batch 怎么拼, 每个请求的随机结果都可复现。

反馈回路 (feedback loop) —— 这一步生成的 token 怎么变成下步输入? 普通 LLM 由引擎自动把上一步 token 喂回去; 但这里一步是 33 个 token, 引擎的自动机制处理不了。作者用一个“待反馈队列”手动接管:

```
# 采样完: 把33路token拼成一行, 算出它的混合embedding, 存进队列
embeds = self.model._prepare_multi_modal_inputs(rows) # 33路->1个向量
# post_process: 把embedding塞进每个请求的待反馈队列
sched_req.data.pending_feedback_queue.append(embeds[row_idx])
# before_decode: 下一步把队列里的embedding直接当decode输入(绕过查表)
weight[:batch_size].copy_(stacked) # 写进一个临时Embedding表
```

### 3.6 阶段三: 声码器与“去延迟” (codec.py)

生成出来的音频码是带延迟错位的, 不能直接解码。 `apply_de_delay_pattern` 把这些错位的行“对齐”回正常的 `[时间, n_vq]` 矩阵, 再交给 MOSS 声码器还原成 24kHz 波形:

```
def apply_de_delay_pattern(delayed_codes):
    # 第idx个码本整体往前挪idx步, 把"楼梯形"错位拉回"矩形"对齐
    for idx in range(n_vq):
        tokens[:, idx] = delayed_codes[idx : idx + T, idx]
    return tokens
```

### 3.7 内容哈希做 radix cache key (request\_builders.py)

SGLang 的 radix cache (前缀缓存) 靠“token id 序列”当 key 来复用 KV cache。但 MOSS 的“token”是 33 维的行, 没法直接当 key。作者给每一行算一个 64-bit 哈希当作虚拟 token id:

```
def build_row_cache_key_ids(rows):
    for row in rows:
        digest = hashlib.blake2b(row.numpy().tobytes(), digest_size=8).digest()
        key_ids.append(int.from_bytes(digest, "little") & ((1 << 63) - 1))
```

这样内容相同的多通道前缀就会得到相同的虚拟 id, radix cache 照常能命中、复用前缀 KV—— 这是“让旧机制适配新数据结构”的经典做法。

## 4. 涉及的知识点 (小白友好版)

### ① 自回归 (Autoregressive, AR) 生成与 KV Cache

**是什么:** 自回归 = “根据已经生成的内容预测下一个 token, 再把它接到后面, 循环往复”。KV Cache 是把每个 token 在注意力里算出的 Key / Value 缓存下来, 下一步就不用重算前面所有 token。

**为什么需要：**没有 KV cache 的话，生成第 1000 个字要把前 999 个字重新算一遍注意力，成本是平方级的；有了它就变线性。

**本 PR 里：**MOSS-TTS 的“音频 token”生成本质也是 AR 的，所以能直接复用 SGLang 的 KV cache 引擎；难点只是“一步多 token”。

## ② RVQ（残差矢量量化）与音频 token / 码本（codebook）

**是什么：**声音是连续波形，神经网络喜欢离散 token。RVQ 把一帧声音用一擦码本逐层逼近：第 1 个码本给个粗略值，第 2 个码本编码“还差多少（残差）”，第 3 个再补……层层逼近，于是 **一帧 = N 个码本 token**。

**为什么需要：**单个码本表达力不够（音质差），多码本残差叠加能在有限词表下逼出高保真音频。

**本 PR 里：** `n_vq=32` 个码本 + 1 个文本通道 = 33 个通道，这正是“一步 33 个 token”的来源。

## ③ Delay Pattern（延迟错位生成）

**是什么：**让第 k 个码本比第 0 个码本晚 k 步开始生成，形成一个“楼梯/三角形”错位。

**为什么需要：**RVQ 里高层码本依赖低层码本的结果（残差关系）。如果同一拍硬要一次预测全部码本，模型还没看到低层就得猜高层，质量差。错位生成让高层码本能“看到”已经定下来的低层码本，更准。（这思路来自 MusicGen / Parler-TTS 等工作。）

**本 PR 里：** `_sample_rows` 里那两行 `pre_audio & post_audio` 布尔掩码，就是把“楼梯”采样区域向量化地画出来；`apply_de_delay_pattern` 在解码前再把楼梯拉平。

## ④ SGLang 的核心组件（Scheduler / Model Runner / Backbone）

**是什么：**SGLang 是高性能 LLM 推理引擎。常见角色：`Scheduler`（决定哪些请求组成一个 batch、何时 prefill/decode）、`Model Runner`（真正调用模型 forward、做采样）、`KV cache 池 / radix cache`（显存与前缀复用）、`tokenizer manager / detokenizer`（文本进出）。

**为什么需要：**把“调度、显存、模型、采样”解耦，才能做连续批处理这种高吞吐优化。

**本 PR 里：**作者复用 SGLang 的 backbone（Qwen3Model）+ KV/radix cache，但自定义了

`OmniScheduler` 接入的 `MossTTSTModelRunner`，重写了 `custom_prefill_forward / before_decode / post_decode / post_process_outputs` 这些钩子，把“33 通道嵌入 + delay 采样 + 反馈队列”插进标准 prefill→decode 循环里。

## ⑤ Continuous Batching（连续批处理）

**是什么：**传统批处理要等一批请求全做完才换下一批；连续批处理是“谁完成谁就退出、空位立刻补新请求”，GPU 永远不空转。

**为什么需要：**不同请求长度差别大，静态批会被最长的请求拖死，GPU 利用率低。



**本 PR 里：**正因为是连续批处理，一个 batch 里各请求的 delay 状态各不相同，所以采样必须向量化地处理 每行不同的状态/采样参数，这就是 `_sample_rows` 大量用 mask 而非 for 循环的原因。

## ⑥ CUDA Graph 与 torch.compile

**是什么：**CUDA Graph 把一连串 GPU kernel 调用“录制”成一张静态图，之后“回放 (replay)”整张图，省掉每步 Python/CPU 发射 kernel 的开销。torch.compile 则是把模型编译成更快的融合算子。

**为什么需要：**小 batch decode 时，CPU 发射 kernel 的开销占比很大，CUDA Graph 能显著降低这部分延迟。

**本 PR 里：**作者预设了 `cuda_graph_bs=[1,2,4,8,16]` 并把 logits 计算移出 forward——因为 CUDA Graph 要求图内张量形状固定、且不能丢失 `customized_info`，所以让被捕获的图只含 backbone，33 路变长的采样放到图外。（PR 说明里提到 CUDA Graph + torch.compile 这次先做 eager 基线，后续提交再上。）

## ⑦ Radix Attention / Radix Cache（前缀缓存）

**是什么：**SGLang 的招牌特性。把所有请求的 token 前缀组织成一棵基数树 (radix tree)，相同前缀的 KV cache 只算一次、大家共享。

**为什么需要：**很多请求开头相同（系统提示、相同参考音频），共享前缀能省大量重复计算与显存。

**本 PR 里：**因为“token”变成了 33 维行，作者用 blake2b 哈希把每行压成一个 64-bit 虚拟 token id（见 3.7），让 radix cache 这套机制无缝适配多通道数据。

## ⑧ 声码器 (Vocoder) / 音色克隆 / RTF / WER

**声码器：**把离散音频 token 还原成可听波形的模型（这里是 MOSS-Audio-Tokenizer，24kHz）。

**音色克隆：**给一段“参考音频”，让合成语音模仿它的音色——本 PR 支持文件路径或 base64 data URI 两种传法，预处理阶段先把参考音频编码成音频 token 当条件。

**RTF (Real-Time Factor, 实时率)：**生成 1 秒音频要花多少秒，越小越好 (<1 才算实时)。PR 里强调 batch=1 是 RTF 最差的情况，所以预处理要多并发把 batch 喂大。

**WER (Word Error Rate, 字错率)：**合成语音再用 ASR 转回文字，和原文比错了多少。3.4 的 padding bug 会拉高 WER。

## ⑨ Monkey-patch / API drift 兼容（一个实用工程技巧）

**是什么：**MOSS 的自定义 Transformers 代码依赖了某些已被改名/删除的 API（如 `PreTrainedConfig`）。作者用一个 `@contextmanager` 临时给 transformers 打补丁，加载完再原样恢复，不污染其他模型。



**启示：**当你必须兼容“别人写死的旧 API”又不想影响全局时，**作用域受限的临时 monkey-patch + 严格回溯**是很干净的解法。

## 5. 我能学到什么 / 延伸思考

### 可复用的工程思想

- **“适配而非重写”**：不去改 SGLang 引擎本身，而是通过 model runner 钩子 + 虚拟 token id 哈希，让特殊模型“假装”成普通模型，复用全部基础设施。这是接入异构模型的高性价比路径。
- **把热路径向量化**：凡是 batch 里逐请求不同的逻辑（delay 状态、每行采样参数），都用布尔 mask / 张量索引一次算完，避免 Python 循环成为瓶颈。冷路径（如 repetition penalty）才允许 per-request 循环。
- **关注点分离 + 流水线并发**：把慢的 CPU 预处理与快的 GPU 生成拆成独立 stage，各自调并发，避免互相拖累。
- **边界值即 bug 温床**：padding 列没切掉就被采样、delay 三角形的 off-by-one——这些都是“形状对齐”留下的隐患，写注释解释“为什么这么切”非常重要。
- **可复现的随机性**：用 (seed, position) 决定每行采样，使结果与 batch 组成无关——这是分布式/批处理推理里调试和测试的刚需。

### 想深入应该补的前置知识

- 音频离散化：EnCodec / SoundStream / RVQ 原理；MusicGen 的 delay pattern 论文。
- SGLang 内部：ForwardBatch / ForwardMode（prefill vs decode）、radix attention 论文、PagedAttention / KV pool。
- CUDA Graph 在推理框架里的使用约束（静态形状、捕获/回放）。
- Qwen3 架构与 SGLang 的权重加载映射（`load_weights` 里的 stacked/packed mapping）。

## #669 [Doc] Add MOSS-TTS in SGLang-Omni docs

文档

gaoyang07

merged 2026-06-03

PR #669

**做了什么：**#609 把 MOSS-TTS 的代码合进来后，本 PR 补上对应的用户文档（cookbook / 索引页），告诉用户如何用

`sgl-omni serve --model-path OpenMOSS-Team/MOSS-TTS-... --config examples/configs/moss_tts.yaml` 启动服务、如何传参考音频做音色克隆、如何用 `{token:N}` 控制时长等。

**价值：**典型的“代码先行、文档跟进”。新功能没有文档等于没上线——这是开源协作里很标准的配套动作，和 #609 是同一主题的延续。

## #668 Remove stale S2-Pro docs test

CI/测试

xinlij

merged 2026-06-03

PR #668

**做了什么：**删除一个已经过时的文档测试文件 `tests/docs/s2pro/test_docs_tts_s2pro.py` 及其空目录。

**价值：**维护性清理。过时的测试会在 CI 里“假报警”或浪费 GPU 资源，及时删掉能让 CI 信号更干净。属于健康的代码库卫生习惯。

## #671 Docs/svg logo（暗色模式适配）

文档

Oxjeffro

merged 2026-06-03

PR #671

**做了什么：**更新文档站的 SVG Logo，使其在暗色模式（dark mode）下也能正常显示。

**价值：**纯前端/品牌呈现的小修复，与推理架构无关，但体现了项目对文档站体验的打磨。

#650 Docs/license

文档



xinlij

merged 2026-06-02

PR #650

**做了什么：**补充 / 更新 License 相关文档。

**价值：**合规性内容，开源项目必备，不涉及代码逻辑。

本日报由自动化流程生成 | 数据源：GitHub API (sgl-project/sglang-omni) | 生成时间：2026-06-03  
讲解深度优先：核心 PR #609 精讲，其余 4 个 PR 简述。代码片段为便于阅读的精简/高亮版，完整代码请见  
对应 PR。